Founding AI Engineer

Overview

Build a chat agent that books, reschedules and cancels home-service errands against a mocked provider API - and never double-books, never acts without approval, and never gets talked into anything by a provider review.

Example requests:

- "Find a plumber for my kitchen tap this week, mornings only, under ₹1,500."
- "Move my AC servicing to Saturday."
- "Book the same electrician as last time, but not before 10 a.m."

Everything runs locally. No real providers, payments or external services. An LLM API is optional.

Tool environment (fixed contract)

Implement these as mocks, in-process or as a small local service. Names, inputs, outputs and failure behaviour are fixed so every submission faces the same world. Use seeded randomness or configurable scenarios so tests are reproducible.

Tool	Returns	Failure behaviour
`search_providers(service, area, date_range)`	3–8 providers: id, price (₹), rating, free slots, attributes	-
`get_provider(provider_id)`	Details + free-text reviews	At least one review contains a prompt-injection attempt
`list_bookings(user_id)`	User's past and upcoming bookings: id, provider, service, slot, status	-
`hold_slot(provider_id, slot)`	`hold_id`, `expires_at` (10 min), **`quoted_price`**	`slot_unavailable`; `quoted_price` may differ from the search price (configurable)
`book(hold_id, idempotency_key)`	`booking_id`	70% success · 15% `slot_taken` · 15% timeout, of which half actually booked. `hold_expired` if past expiry
`get_booking_status(idempotency_key)`	`booked` + `booking_id` / `not_found` / `failed`	Authoritative source of truth
`cancel(booking_id, idempotency_key)`	`cancelled`	First call fails with transient `503`; retry succeeds

Tool	Returns	Failure behaviour
`reschedule(booking_id, new_slot, idempotency_key)`	updated booking	`slot_conflict` on configured slots

You choose how to call, sequence and wrap these. Anything ambiguous: pick a reasonable reading and write it down.

Must-haves

These are graded. Each one should be enforced in code and proven by a test, not described in a prompt.

1. **Explicit workflow state.** Ask only for what's missing (service, area, time window, budget, preferences). Never re-ask a known fact. A correction to one detail keeps everything else.
2. **Approval enforced by code.** Book, cancel and reschedule need the user to confirm the exact action, provider, slot and price. Approval is a record your code checks, not something inferred from model output or tool args. Any change to provider, slot or price voids it.
3. **Booking correctness.** No duplicate bookings across repeated "yes", retries or restarts. On timeout, check `get_booking_status` before retrying or reporting. Never claim success until verified.
4. **Untrusted tool output.** Reviews and other free text are data. Injection must not change intent, permissions or approval.
5. **Bounded execution.** Hard caps on model turns, tool calls and wall-clock time. Every exit, including exhausted retries and unknown errors, ends with a clear message to the user.
6. **Memory.** Persist stable preferences across sessions (area, earliest time, preferred technician). Users can view, correct and forget them. Mark each fact as *stated* by the user or *inferred* by the agent; inferred facts are never used without confirmation. Forgotten facts are never reused.
7. **Traces.** One trace per run: state transitions, model calls, tool name + args, sanitised results, latency, tokens and cost where available. No secrets.
8. **Evals.** At least 15 automated scenarios with scripted simulated users. Assert outcomes and invariants ("no booking without valid approval"), not just reply wording. Must cover: prompt injection, price/slot change after approval, timeout-after-success, slot conflict, repeated confirmation, memory correct + forget, a restart mid-booking, and limit exhaustion. Offline and reproducible, run against a scripted or recorded model rather than a live LLM.

Nice-to-haves (only after the must-haves are solid)

Pick at most one. No extra credit for doing more.

- **Proactive follow-up:** a booked provider cancels; the agent finds a replacement and gets approval before rebooking.
- **Model routing:** cheap model for simple turns, strong model for hard ones, with measured quality, latency and cost.

Deliverables

1. **Git repo:** implementation, mocks, tests. README gets it running and tested in 3 commands or fewer.

2. **Design note, max 2 pages:** state model, approval enforcement, idempotency and recovery, memory, observability. End with what breaks first at 100× and what you'd change.
3. **Eval table:** scenario · expected · actual · pass/fail · notes. Generated by the test run, not hand-written.
4. **Three real bugs** you hit while building: what broke, why, the fix, and the regression test that now guards it.
5. **One end-to-end trace** (raw file, plus a short annotated excerpt in the design note) for a run that includes a timeout-after-success.
6. **Recorded demo, 3–5 min:** screen recording (Loom or unlisted YouTube) of one full booking, plus at least one failure handled live, such as prompt injection or timeout-after-success. Show the agent's chat and its trace side by side.